

J.A.R.V.I.S. — Phase 3 (v3)

System Design Document | Cognitive Architecture + Modular Monolith

Phase 1 Planning	Phase 2 Requirements	Phase 3 Design	Phase 4 Implementation	Phase 5 Testing	Phase 6 Deployment
------------------	----------------------	----------------	------------------------	-----------------	--------------------

1. Architecture Pattern: Modular Monolith

Pattern	Modular Monolith — single process, layered modules, clean interfaces
Single process	python jarvis.py starts everything. One process, two threads (main + wake word daemon).
Layer order	core/ coordinates input/, output/, tools/. No tool imports another tool. No input/output imports from tools/.
LLM boundary	Only core/llm.py imports anthropic. All other modules call ask() or ask_simple().
Voice boundary	Only output/voice.py calls ElevenLabs/pyttsx3. All modules call speak(text).
Memory boundary	Only core/memory.py and core/knowledge_graph.py access Mem0/ChromaDB/NetworkX.
Tool interface	Every tools/ module exposes exactly one public function: execute(inputs: dict) -> str
Adding a tool	Create tools/new_tool.py, add schema to TOOL_DEFINITIONS, add to TOOLS registry. Zero other files change.

2. System Architecture Layers

Input	Wake word (Porcupine offline) STT (Whisper local, webrtcvad VAD) Text input Screen reader
Orchestrator	CEO agent — intent parsing, tool routing, memory assembly, context budget enforcement
LLM Core	Claude API wrapper — tool_use, retry logic, token logging, ask() / ask_simple()

Cognitive	8-system memory: Mem0 (semantic+associative) Episodic (SQLite) Procedural (skill library) Knowledge graph (NetworkX) Confidence scoring Metacognitive map Consolidation Forgetting curve
Tools	pc_control web_search trading_tool email_tool coding_tool — each independently testable
Output	Voice (ElevenLabs + pyttsx3 fallback) Display (colorama terminal) Notifications (plyer toast)

3. Cognitive Architecture Design

Critical architecture note — what 'learning' means: The LLM (Claude, GPT-4o, Mistral) never changes. Its weights are frozen and trained by its provider. What improves over time is the *context injected into every API call* — richer notes, better-organised facts, deeper domain knowledge retrieved from the memory systems below. This pattern is called RAG (Retrieval Augmented Generation). Every major AI product (Perplexity, Cursor, GitHub Copilot, NotebookLM) uses it. The LLM is the cortex — general intelligence, frozen. The memory architecture is the hippocampus — experience, accumulated over time.

J.A.R.V.I.S. implements a cognitive architecture modelled on the human brain. The human brain is not one memory system — it is eight distinct systems operating simultaneously, each with different storage mechanisms, retrieval triggers, and decay rates. This architecture replicates all eight using purpose-built AI libraries and custom components. The result is the closest any software system can come to human-like learning, recall, and knowledge accumulation without modifying the underlying LLM weights.

3a. The 8 memory systems

1. Semantic memory <i>General world knowledge — facts without personal context</i>	ChromaDB vector store + NetworkX/Neo4j knowledge graph. Stores facts, concepts, relationships. Retrieved by semantic similarity search. This is the foundation — everything else builds on top of it. Status: Already planned (v1 core/memory.py + v2 knowledge_graph.py)
2. Episodic memory <i>Personal experiences anchored in time and sequence</i>	SQLite episodic store. Every agent action is logged with: timestamp, agent_id, task_id, action_type, inputs, outputs, success_score. Answers 'what happened, when, and what was the outcome?' This is what makes agents feel like they have a genuine history. Status: New — add to v2 Phase 1 (core/episodic_store.py)
3. Procedural memory <i>How to do things — reusable skills and patterns</i>	Skill library (JSON + Python). After 5+ similar tasks, the consolidation job extracts a reusable pattern and stores it as a named skill. Future tasks load the skill as a prompt prefix. Agents stop re-figuring-out things they have done many times. Status: New — add to v2 Phase 1 (core/skill_library.py)

4. Working memory <i>Active conscious focus — the 4–7 items currently in mind</i>	Context window (200k tokens) + rolling deque(maxlen=20) conversation buffer. Limited, volatile, fast. The context budget governs what fits: system prompt (2k) + memories (3k) + conversation (10k) + task (5k) + tools (8k). Status: Already planned (v1 core/memory.py deque buffer)
5. Associative memory <i>Spreading activation — related concepts light up together</i>	Knowledge graph traversal. When retrieving node X, also retrieve its 1-hop and 2-hop neighbours. 'JWT' activates 'authentication' activates 'bcrypt' activates 'the security bug from last month.' Enables non-obvious connections that pure similarity search cannot make. Status: New — add to v2 Phase 1 (core/knowledge_graph.py spreading_recall())
6. Prospective memory <i>Intentions about the future — planned actions</i>	APScheduler + SQLite task queue. Tasks have a scheduled_at timestamp. 'Check CVE feed every Friday at 9am.' 'Run memory consolidation nightly at 2am.' 'Review Junior Dev output every time a PR is created.' Agents remember to do things without being reminded. Status: Already planned (v2 core/task_queue.py + APScheduler)
7. Confidence memory <i>Emotional signals — difficulty and success scoring</i>	Activation scoring system. Every memory has a float confidence_score (0.0–1.0). Score spikes on successful retrieval and use. Score decays daily (Ebbinghaus curve). Low-confidence memories trigger the agent to research before acting. High-confidence memories are acted on directly. Status: New — add to v2 Phase 1 (metadata field on all ChromaDB entries)
8. Metacognitive memory <i>Knowing what you know — competence self-awareness</i>	Per-agent competence map (JSON). Each agent maintains topic coverage scores: {'FastAPI': 0.9, 'cryptography': 0.3, 'React': 0.7}. Score below 0.5 triggers autonomous research before starting the task. Score above 0.8 allows direct execution. Agents know their own limits. Status: New — add to v2 Phase 2 (agents/base_agent.py competence_map)

3b. Key cognitive mechanisms

Mem0	Mem0 (pip install mem0ai) is the unified memory API layer. It wraps ChromaDB, the knowledge graph, and structured memory in a single interface: <code>memory.add(text, agent_id='senior_dev')</code> , <code>memory.search(query, agent_id='senior_dev')</code> , <code>memory.get_all(agent_id='senior_dev')</code> . Mem0 automatically extracts what is worth remembering from conversations without manual [REMEMBER:] tagging — it uses its own extraction LLM call. It is used in production by major AI products and is actively maintained. Mem0 handles semantic + associative memory. Episodic, procedural, confidence, and metacognitive memory are built on top as custom modules.
Knowledge graph	The knowledge graph stores entities and named relationships between them. Node types: Agent, File, Concept, Bug, Project, KnowledgeSource, Skill. Edge types: BUILT, LEARNED, LEARNED_FROM, FOUND, FIXED, DEPENDS_ON, CALLS, BELONGS_TO, USED_IN, REVIEWED, TAUGHT, HAS_BUG. Technology: NetworkX (v1/early v2, in-memory, persisted to JSON) → Neo4j (late v2, full graph database, Cypher query language). Populated automatically: every agent action writes edges. Static analysis (Python ast module) extracts CALLS relationships from code files. Spreading activation retrieves 2-hop neighbours on every query.
Memory consolidation	Memory consolidation is the most important brain process missing from most AI systems. In the human brain, the hippocampus consolidates memories during sleep — compressing raw episodic experiences into semantic facts stored in the cortex. For Jarvis, a nightly background job runs the same process: it reads today's episodic log, asks the LLM to extract durable facts, stores them in ChromaDB and the knowledge graph, applies the forgetting curve decay to all existing memories, and prunes memories whose activation score has fallen below the threshold. Agents wake up the next day with distilled knowledge rather than a growing pile of raw logs.
Forgetting curve	Ebbinghaus (1885) mapped human forgetting mathematically: $R = e^{(-t/S)}$ where t is elapsed time and S is memory strength. Implemented in Jarvis as: every memory has a float activation score starting at 1.0. Score decays by 5% daily. Score spikes by +0.3 on every retrieval (reinforcement). Memories below 0.1 are archived to cold storage (not deleted — can be manually recalled). This means agents naturally retain frequently-used knowledge and let rarely-used knowledge fade — exactly like humans.
Spreading activation	When the knowledge graph retrieves a node, it also retrieves all nodes within 2 hops. Query 'JWT authentication' → activates JWT node → spreads to: <code>auth.py</code> (USED_IN), <code>bcrypt</code> (RELATED_TO), <code>Bug#7</code> (FOUND_IN <code>auth.py</code>), <code>Senior_Dev_Agent</code> (BUILT <code>auth.py</code>). All activated nodes are injected into the LLM context. The LLM sees a richer, more connected picture — not just a single matching fact but an entire related cluster. This is what makes Jarvis feel associative rather than merely database-like.

Context management	Context management is the engineering problem that determines whether the cognitive architecture works in practice. Every LLM API call has a fixed token budget. The <code>build_context()</code> function in <code>core/memory.py</code> assembles the prompt in priority order: (1) system prompt [2,000 tokens], (2) top-3 semantic memory recall [1,500 tokens], (3) spreading activation neighbours [1,500 tokens], (4) episodic summary for current task [1,000 tokens], (5) active project record [1,500 tokens], (6) task context object [5,000 tokens], (7) conversation history deque [10,000 tokens], (8) current user input [1,000 tokens]. Total: ~23,500 tokens — well within the 200k limit with room for tool results. If any section overflows its budget, it is compressed by a summarisation LLM call, never truncated blindly.
---------------------------	---

4. Project Folder Structure

<code>jarvis/</code>	Root — single deployable unit (modular monolith)
<code>jarvis.py</code>	Main entry point — composition root
<code>.env</code>	All API keys — never committed
<code>requirements.txt</code>	All pinned dependencies
<code>config.json</code>	User preferences, allowed apps, competence maps
<code>README.md</code>	Setup guide, Gmail App Password walkthrough
<code>core/</code>	System brain — coordinates all layers
<code>orchestrator.py</code>	Command routing, tool dispatch, confirmation gates
<code>llm.py</code>	Claude API wrapper — only file importing anthropic
<code>memory.py</code>	Mem0 unified API + deque buffer + context assembly
<code>knowledge_graph.py</code>	NetworkX graph — relationships, spreading activation
<code>episodic_store.py</code>	SQLite episodic log — timestamped agent events
<code>skill_library.py</code>	Procedural memory — reusable task patterns
<code>consolidation.py</code>	Nightly job — compress episodes → semantic facts
<code>context_budget.py</code>	Token counting (tiktoken) + budget enforcement
<code>input/</code>	Hearing layer
<code>wake_word.py</code>	Porcupine daemon thread
<code>speech_to_text.py</code>	Whisper + webrtcvad VAD
<code>output/</code>	Speaking layer
<code>voice.py</code>	ElevenLabs + pyttsx3 fallback
<code>display.py</code>	colorama terminal + ptyer notifications
<code>tools/</code>	Action layer — each independently testable
<code>pc_control.py</code>	Apps, files, keyboard (pyautogui, psutil)
<code>web_search.py</code>	DuckDuckGo search + browser
<code>trading_tool.py</code>	Prices + news (yfinance, Alpha Vantage)
<code>coding_tool.py</code>	Code Q&A; + file review
<code>email_tool.py</code>	Gmail SMTP with confirmation gate

data/	Persistent storage
chroma_db/	ChromaDB vector store (Mem0 backend)
episodic.db	SQLite episodic memory log
knowledge_graph.json	NetworkX graph persistence
skill_library.json	Procedural memory patterns
logs/	Session logs, LLM call logs, email logs
tests/	Unit test stubs — one per module

5. Technology Stack

Component	Technology
Language + Architecture	Python 3.11 · Modular Monolith
LLM brain	Anthropic Claude API (tool_use)
Wake word	Picovoice Porcupine (offline)
Speech-to-text	OpenAI Whisper (local) + webrtcvad
Voice output	ElevenLabs (online) + pyttsx3 (offline fallback)
Unified memory API	Mem0 (semantic + knowledge graph + structured)
Vector store	ChromaDB (local, Mem0 backend)
Knowledge graph	NetworkX (v1) → Neo4j Community (v2)
Episodic memory	SQLite + custom episodic_store.py
Procedural memory	skill_library.py + JSON persistence
Memory consolidation	APScheduler nightly job + LLM extraction
Forgetting curve	Custom activation scoring (float metadata on ChromaDB)
Context budget	tiktoken token counting + build_context() assembly
PC automation	pyautogui + psutil + os + subprocess
Web search	duckduckgo-search (no API key)
Trading data	yfinance + Alpha Vantage
Email	smtplib + Gmail SMTP App Password
Terminal output	colorama (ANSI on Windows)

Notifications	plyer (Windows toast)
Secrets	python-dotenv (.env)
Testing	pytest + pytest-mock